

实验2：内核printf与清屏功能实现

框架文件速查

本实验任务	对应文件	TODO标注
任务1：printint 算法	kernel/driver/console.c	TODO [Lab2-任务1]
任务2：printf 主体	kernel/driver/console.c	TODO [Lab2-任务2]
任务3：clear_screen	kernel/driver/console.c	TODO [Lab2-任务3]
任务4：加入构建	Makefile (SRCS 添加一行)	—

实验目标

在实验1中，我们已经成功向串口（屏幕）输出了固定的字符串 "Hello OS"。但在真实的内核开发与调试中，我们需要能够打印变量的值（比如地址 0x80000000，报错码 -1 等）。本实验的目的在于：在没有 C 标准库 <stdio.h> 的支持下，利用 C 语言的可变参数机制（stdarg.h）和数学运算，自己手写一个极其简化的内核 printf 函数。并且，利用 ANSI 转义控制符，实现华丽的“清屏”效果。

理论补充：为什么自己写 printf 这么难？

平常我们在写应用层代码时，调用 printf("value=%d", 123) 似乎理所当然。但在操作系统的内核里，我们要面对三个棘手的底层问题：

- 参数数量不固定：**printf 可能收到 1 个参数，也可能收到 5 个参数，内核怎么知道你传了多少个参数进来？（需要用到 stdarg.h 提供的 va_list 魔法）。
- 硬件只认字符：**串口硬件 uart_putc 只能一次发送一个 ASCII 字符。如果你要打印数字 123，硬件是不认识一百二十三的。你必须通过数学运算（取余和除法），把 123 拆解成字符 '1'，'2'，'3'，然后逐个发给硬件。
- 进制转换问题：**操作系统调试时，打印普通的十进制 %d 远远不够，打印内存地址通常需要十六进制 %p 或 %x。

本实验的核心任务，就是把上述过程手工实现一遍。

★ 可变参数原理速成

stdarg.h 在裸机环境下依然可用——它是由编译器（而非 C 标准库）提供的内置功能，不依赖操作系统。

可变参数的底层原理：函数参数从左到右依次压栈（或放在寄存器中），va_list 本质上是一个指针，指向“最后一个固定参数”紧邻的内存区域，然后通过 va_arg 按照你指定的类型不断往后移动这个指针来读取参数：

1 调用 printf("val=%d %s", 42, "hi") 后，栈的示意：



四个关键宏：

宏	作用
<code>va_list ap</code>	声明参数列表指针
<code>va_start(ap, fmt)</code>	初始化 ap，使其指向 fmt 之后的第一个可变参数
<code>va_arg(ap, type)</code>	取出当前 ap 指向的参数（按 type 类型），然后 ap 移向下一个
<code>va_end(ap)</code>	结束可变参数访问（必须调用，否则行为未定义）

任务指南与实施步骤

任务1：实现数字转字符串的算法（`printint`）

应对难题 2 和 3：如何把数字 123 变成屏幕上的 "123"？

核心思路是进行“余数截取”。比如 123 除以 10，余 3，我们就得到了最后一位字符 `'3'`；接着 $123 // 10 = 12$ ，再用 12 除以 10 余 2，得到字符 `'2'`；以此类推。需要注意的是，这样解析出来的字符是反向的（先出3、再出2、最后出1），我们需要把它们倒过来输出。

★ 算法追踪示例（以打印 -123 为例）

```
1  输入：xx = -123, base = 10, sign = 1 (有符号)
2
3  ① sign && xx<0 成立, x = 123, sign = 1 (记录要加负号)
4
5  ② do-while 循环 (x 从 123 开始) :
6      第1次: buf[0] = digits[123 % 10] = digits[3] = '3', x = 12
7      第2次: buf[1] = digits[12 % 10] = digits[2] = '2', x = 1
8      第3次: buf[2] = digits[1 % 10] = digits[1] = '1', x = 0 (退出循环)
9
10 ③ sign 为1, 补充负号: buf[3] = '-', i = 4
11
12 ④ 倒序输出 buf[3..0]: '-', '1', '2', '3' → 屏幕显示 -123 ✓
```

目标文件：`kernel/driver/console.c`

找到 `TOD0 [Lab2-任务1-步骤2-A]` 和 `TOD0 [Lab2-任务1-步骤2-B]`，填写核心循环：

```

1 // 步骤A: do-while 循环 (把 (void)b; 那行删掉)
2 do {
3     buf[i++] = digits[x % base];
4     x = x / base;
5 } while (x != 0); // 把 while(0) 改成 while(x != 0)
6
7 // 步骤B: 倒序输出 (把空行替换为)
8 while (--i >= 0)
9     uart_putc(buf[i]);

```

你的任务:

在项目中修改 `kernel/driver/console.c` , 填写上述 TODO。

核心代码框架参考:

```

1 // 定义一个用来把十六进制数字快速映射到字符的查表数组
2 static char digits[] = "0123456789abcdef";
3
4 // 这是一个极其关键的基础函数: 将任意进制的数字转为字符输出
5 // xx: 要打印的数字; base: 进制(通常是10或16); sign: 是否有符号数(处理负数用)
6 static void printint(int xx, int base, int sign) {
7     char buf[16]; // 临时缓冲区, 用来存放倒序的字符
8     int i;        // 缓冲区指针
9     uint32_t x;   // 无符号版本的数字
10
11     // 如果有符号十进制, 且值为负数
12     if(sign && (sign = xx < 0))
13         x = -xx; // 把负数转为正数处理
14     else
15         x = xx;
16
17     i = 0;
18     // 核心算法: 循环对 base 取余, 存入 buf, 直到 x 为 0
19     do {
20         // [填空尝试]: 请在这里补充 x % base 以及 x = x / base 的逻辑,
21         // 将结果当作下标从 digits 数组中取出字符, 并存入 buf[i++]
22     } while(x != 0);
23
24     // 如果原来是负数, 要补上负号 '-'
25     if(sign) {
26         buf[i++] = '-';
27     }
28
29     // [填空尝试]: 此时 buf 里的字符是倒着的 (比如 123 存成了 '3', '2', '1'),
30     // 请写一个 while 循环, 从 i-1 开始逆遍历直到 0,
31     // 依次调用 uart_putc(buf[i]) 把它正确输出到屏幕。
32 }

```

任务2：解析变长参数，完成终极 `printf`

应对难题 1：如何处理未知数量的参数？

📁 目标文件： `kernel/driver/console.c`

找到 `TOD0 [Lab2-任务2-步骤A/B/C/D]`，依次填写各格式符的处理逻辑：

```
1 case 'd':
2     printint(va_arg(ap, int), 10, 1);    // 步骤A：有符号十进制
3     break;
4 case 'x':
5     printint(va_arg(ap, int), 16, 0);    // 步骤B：十六进制
6     break;
7 case 'p':
8     printint(va_arg(ap, unsigned long), 16, 0); // 步骤C：地址（64位无符号）
9     break;
10 case 's':
11     if ((s = va_arg(ap, char*)) == 0) s = "(null)";
12     for (; *s; s++) consputc(*s);        // 步骤D：逐字符输出
13     break;
```

这里需要引入 C 编译器内置的 `<stdarg.h>` 库（即使在裸机环境下编译器也自带这个功能）。它提供了四个核心宏指令：`va_list`（声明参数列表），`va_start`（开始读取参数），`va_arg`（拿取下一个参数），`va_end`（结束读取）。

🔧 你的任务：

实现 `printf` 的主循环框架。

```
1 #include <stdarg.h>
2
3 // 提前声明你的单字符输出函数
4 extern void uart_putc(char c);
5 // 如果你想实现完整的打印，你还需要实现一个简单的单字符输出函数
6 void consputc(int c) {
7     uart_putc(c);
8 }
9
10 // 你的主角：简易版 printf
11 void printf(char *fmt, ...) {
12     va_list ap;          // 定义一个变量 ap，用来跟踪附加的参数
13     int i, c;            // i：用于遍历格式字符串 fmt
14     char *s;             // 用于指向提取的字符串参数
15
16     va_start(ap, fmt);   // 告诉 ap，从 fmt 参数的后面正式开始读取变长参数
17
18     for(i = 0; (c = fmt[i] & 0xff) != 0; i++){
19         // 如果不是 '%', 那就是普通字符，直接原样输出！
20         if(c != '%'){
21             consputc(c);
```

```

22         continue;
23     }
24
25     // 如果遇到了 '%', 说明后面的字符是控制符, 我们要进行特殊处理
26     c = fmt[++i] & 0xff;
27     if(c == 0) // 如果 '%' 是字符串最后一个字符, 说明传错了, 直接退出
28         break;
29
30     // 根据 % 后面的字母, 决定把后面的变长参数当作什么类型取出来
31     switch(c){
32     case 'd':
33         // 拿到参数, 并告诉它以十进制、有符号数的方式打印
34         printint(va_arg(ap, int), 10, 1);
35         break;
36     case 'x':
37     case 'p':
38         // 拿到参数, 以十六进制、无符号的方式打印
39         // [填空尝试]: 请仿照 %d, 调用 printint 这里填入逻辑
40         break;
41     case 's':
42         // 拿到一个字符串指针
43         if((s = va_arg(ap, char*)) == 0)
44             s = "(null)"; // 鲁棒性: 防止空指针崩溃
45         // [填空尝试]: 写一个循环, 逐字符输出字符串 s
46         break;
47     case '%': // 如果是 '%%', 则是指真的想打印一个 '%' 号
48         consputc('%');
49         break;
50     default:
51         // 遇到不认识的控制符 (比如 %f, 当前小系统不支持浮点), 打印错误
52         consputc('%');
53         consputc(c);
54         break;
55     }
56 }
57 va_end(ap); // 结束参数读取
58 }

```

任务3: 终端黑魔法 —— 实现清屏与光标复位

在操作系统的命令行界面 (Shell) 中, 你一定用过 `clear` 命令。在裸机下, 我们并不需要真的去操作显卡缓存才能清屏, 现在的终端仿真器 (比如 QEMU 中挂载的终端) 都支持一种叫做 **ANSI 转义序列 (ANSI Escape Codes)** 的魔法。只要按特定的格式向终端输出一段字符串, 终端软件就会帮你执行对应的特效。

常见魔法咒语:

- 清空整个屏幕的内容: `\x1b[2J`
- 把光标移动回屏幕正左上角 (0, 0) 的位置: `\x1b[H`

- 改变字符的颜色为红色： `\x1b[31m` (+随后要打印的内容) + `\x1b[0m` (恢复默认颜色)

注： `\x1b` 代表 ASCII 中的 ESC 键（十进制的 27）。

 目标文件： `kernel/driver/console.c`

找到 `TOD0 [Lab2-任务3]`，填写清屏逻辑：

```
1 void clear_screen(void) {
2     printf("\x1b[2J");    // 清除整个屏幕
3     printf("\x1b[H");    // 光标归位到左上角
4 }
```

 你的任务：

写一个优雅的清屏函数。并在 `main.c` 启动测试时调用它。

```
1 void clear_screen() {
2     // 提示：你已经写好了 printf 啦！直接用吧！
3     // [填空尝试]：在这里调用极简的两次 printf，一次发清屏咒语，一次发光标复位咒语
4 }
5
6 // 在 main.c 进行最终合体验收：
7 void start_main() {
8     clear_screen(); // 验收：屏幕瞬间变干净！
9     printf("Welcome to WHU OS Lab!\n");
10    // 验收：这三个特殊的占位符必须被正确解析和打印！
11    printf("Kernel loaded at address: %p\n", 0x80000000);
12    printf("Signed integer test: %d\n", -123);
13    printf("String test: %s\n", "Hello, Variadic Parameters!");
14
15    while(1);
16 }
```

任务4 — 将 `console.c` 加入构建并更新 `main.c`

 更新 `Makefile`

打开 `Makefile`，在 `SRCS` 列表中追加新文件：

```
1 SRCS = \
2     kernel/boot/entry.S \
3     kernel/driver/uart.c \
4     kernel/boot/main.c \
5     kernel/driver/console.c    # ← Lab2 新增
```

 更新 `kernel/boot/main.c`

将原来的 `start_main()` 从调用 `uart_puts` 升级为使用新的 `printf`：

```
1  /* Lab2 完成后, main.c 应该是这样 */
2  void start_main() {
3      clear_screen();
4      printf("Welcome to WHU OS Lab!\n");
5      printf("Kernel loaded at: %p\n", 0x80000000);
6      printf("Signed int test: %d\n", -123);
7      printf("String test: %s\n", "Hello, Variadic Parameters!");
8      while(1);
9  }
```

运行验收：

```
1  make run
```

预期看到屏幕清空后，打印出4行信息，地址、负数、字符串均正确显示。

避坑与故障排除策略 (Debugging)

如果你的 `printf` 输出的数字是乱码，或者系统直接崩溃 (Kernel Panic)，请按照以下顺序检查：

1. **查表数组写错了吗？** `printint` 中的余数如果是 `10`，要在 `digits` 数组里找到第 10 个元素 `'a'`。如果你忘了在 `printint` 补上倒序排列输出的循环，你打出来的 `123` 会在屏幕上变成倒着的 `321`！
2. **格式字符串是不是用错了变量？** GDB 调试时，重点在这个位置打断点： `b printf`。然后通过命令 `info args` 查看 `fmt` 的字符串是否正确传入了函数。
3. **负数问题！** 负数转正数最容易引发一个经典 bug：如何处理整数的极小值 `INT_MIN` (`-2147483648`)？如果直接写 `x = -xx` 会溢出！在这个极简版的内核中，暂时先不管溢出，如果想要完美实现（如 xv6 那样强制使用 `uint32_t` 接管转换），需要额外仔细考虑无符号转化的问题。

进阶实验验收题

实验完成后需能回答以下问题：

1. 请观察并思考：你现在用的是 `printint` 这种死循环求余数的除法方式来获得各个位数的字符的。你知道在底层的硬件架构中，某些早期的或者精简版 CPU 往往**不支持原生的除法指令** `div` 吗？如果我们遇到一个不支持除法指令的裸机架构，这个通过除法来实现的 `printf` 该怎么被重写或规避？
 2. ANSI 转义符除了能控制清屏，还能控制颜色。如果让你把内核发生的严重报错以**红色高亮**（且后续的正常打印自动恢复回白色）的形式打印出来，你需要定义怎样的便捷宏指令来实现？
-

★ 本实验完成后的全貌

Makefile SRCS 状态:

```
1 SRCS = entry.S + uart.c + main.c + console.c    # Lab2之后
```

`start_main()` 当前形态 (Lab3 完成后会继续扩展) :

```
1 void start_main() {  
2     clear_screen();  
3     printf("Welcome to WHU OS Lab!\n");  
4     printf("Kernel loaded at: %p\n", 0x80000000);  
5     printf("Signed int: %d\n", -123);  
6     printf("String: %s\n", "Hello!");  
7     while(1);  
8 }
```

Lab3 预告: 下一个实验你将实现 `kalloc.c` 和 `vm.c` , 届时 `start_main()` 会在 `clear_screen()` 之前增加内存初始化调用: `kinit()` 和 `kvmininit()` 等。